

Please provide an example of a sequential agent using google-adk 1.5 and a separate code to test it

In the Google Agent Development Kit (ADK), a `SequentialAgent` is an orchestration primitive that executes an array of sub-agents one after another in a deterministic assembly-line structure. They communicate through a shared invocation context (`InvocationContext`), storing outputs via an `output_key` so that subsequent agents can consume them.

Here is an end-to-end example implementing a pipeline that analyzes a raw user request, converts it into a structured insight, and generates a tailored response, accompanied by an integration test file.

1. Defining the Agent (`agent.py`)

In this module, we set up two `LlmAgent` instances. The first parses the core intent and stores it under `extracted_data`, and the second leverages that output key to generate the final response. Both are orchestrated linearly inside a `SequentialAgent`.

Python



```
# agent.py
from google.adk.agents import LlmAgent, SequentialAgent

# Step 1: Extract core product metrics/attributes from user text
extractor_agent = LlmAgent(
    name="ExtractorAgent",
    model="gemini-2.5-flash",
    instruction="""
    Analyze the user input and extract any product mentions, specific issues, or scores.
    Output only a clean JSON-like structure or structured summary of the extraction.
    """,
    output_key="extracted_data"
)

# Step 2: Use the extracted data to craft a final, professional response
responder_agent = LlmAgent(
    name="ResponderAgent",
    model="gemini-2.5-flash",
    instruction="""
    Review the structured information inside {extracted_data}.
    Generate a polished, professional email response addressing the user's issue based on the extracted data.
    """
)
```

```

        """
        output_key="final_response"
    )

# Define the Sequential Pipeline
sequential_pipeline = SequentialAgent(
    name="SupportPipeline",
    sub_agents=[extractor_agent, responder_agent]
)

```

2. Testing the Agent (test_agent.py)

To test the sequence locally, mock an `InvocationContext` containing the starting payload. We then loop over the async generator returned by `run_async` and inspect the modified state.

Python



```

# test_agent.py
import asyncio
from google.adk.context import InvocationContext
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing test environment...")

    # Create an invocation context and supply the initial user input
    ctx = InvocationContext()
    ctx.session.state["user_input"] = (
        "Hey! The performance metric on my dashboard is sitting at 65%. "
        "It feels sluggish, and I'm using the Premium Tier package."
    )

    print("\nExecuting Sequential Pipeline...")
    # Run the sequential agent through its pipeline array
    async for event in sequential_pipeline.run_async(ctx):
        # Optional: You can monitor stream events here if required
        pass

    print("\n--- Pipeline Execution Completed ---")

    # Validate Step 1 output
    extracted = ctx.session.state.get("extracted_data")
    print(f"\n[Step 1 Output] Extracted Data:\n{extracted}")

    # Validate Step 2 output

```

```

final_output = ctx.session.state.get("final_response")
print(f"\n[Step 2 Output] Final Response:\n{final_output}")

# Simple assertions to ensure state was populated correctly
assert extracted is not None, "Pipeline failed: Extracted data is empty."
assert final_output is not None, "Pipeline failed: Final response is empty."
print("\nTest Status: SUCCESS")

if __name__ == "__main__":
    asyncio.run(run_pipeline_test())

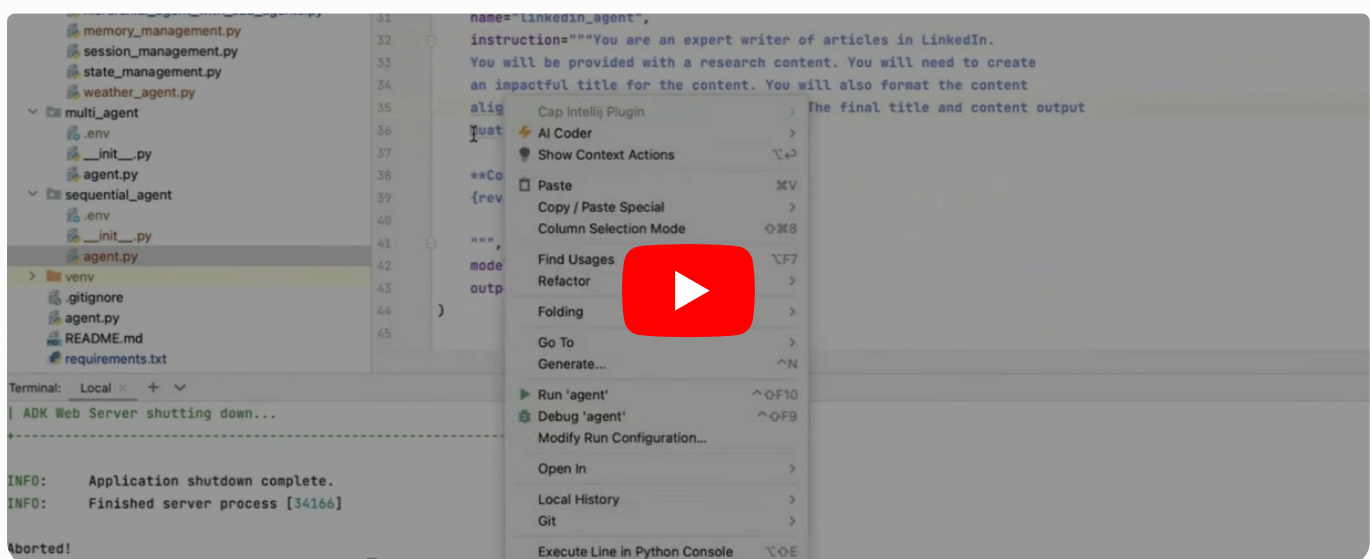
```

Key Takeaways

- **Shared State Transfer:** Notice how `responder_agent` references `{extracted_data}` directly in its prompt string instructions. ADK resolves variables wrapped in braces using keys found inside `ctx.session.state`.
- **Deterministic Execution:** The `SequentialAgent` steps through `sub_agents` explicitly matching the array index tracking sequence (`[extractor_agent, responder_agent]`), avoiding uncoordinated multi-agent racing conditions.

For a deeper dive into designing complex multi-agent architectures using Google ADK, you can check out this guide on [Building a sequential agent using Google ADK](#), which covers step-by-step pipeline configurations and state handling principles.

📺 Building a sequential agent using Google ADK. - YouTube
Rajib Deb · 459 views



Instead of a single `user_input` rewrite the code for a list of 3 example `user_input` to test `run_pipeline_test` 3 times

Here is the updated test file rewritten to iterate through a list of multiple sample user inputs. We can loop over the test cases, instantiate a clean `InvocationContext` for each run, and print out the distinct pipeline transformations.

Updated Testing Code (`test_agent.py`)

Python



```
# test_agent.py
import asyncio
from google.adk.context import InvocationContext
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing multi-input pipeline test environment...\n")

    # List of 3 sample user inputs representing different scenarios
    test_inputs = [
        "Hey! The performance metric on my dashboard is sitting at 65%. It feels slug",
        "Our API integration is throwing 500 Internal Server errors during checkout.",
        "I'm just exploring the Free Trial, but I can't seem to find where to export
    ]

    # Iterate through each sample input
    for index, user_input in enumerate(test_inputs, start=1):
        print(f"=====")
        print(f" RUNNING TEST CASE #{index}")
        print(f"=====")
        print(f"Raw Input: '{user_input}'\n")

        # Create a fresh context for each individual run to avoid state leakage
        ctx = InvocationContext()
        ctx.session.state["user_input"] = user_input

        print("Executing Sequential Pipeline...")
        # Stream the sequential agent execution
        async for event in sequential_pipeline.run_async(ctx):
            pass # Monitoring execution stream if needed

        print("Pipeline Execution Completed.")
```

```

# Extract and verify the state modifications
extracted = ctx.session.state.get("extracted_data")
final_output = ctx.session.state.get("final_response")

print(f"\n[Step 1 Output] Extracted Data:\n{extracted}")
print(f"\n[Step 2 Output] Final Response:\n{final_output}")

# Assertions to ensure each run successfully completed its lifecycle
assert extracted is not None, f"Pipeline failed on case #{index}: Extracted d
assert final_output is not None, f"Pipeline failed on case #{index}: Final re

print(f"\nTest Case #{index} Status: SUCCESS\n")

if __name__ == "__main__":
    asyncio.run(run_pipeline_test())

```

What Changed?

- **State Isolation:** The code creates a new instance of `InvocationContext()` inside the loop for each test string. This ensures that data from the previous iteration does not bleed over or accidentally fulfill prerequisites for the next agent run.
- **Batch Verification:** The assertion checks execute sequentially for each input, guaranteeing that all three distinct scenarios satisfy the pipeline's output criteria.

Please replace `test_inputs` with `sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]`

Here is the updated code replacing `test_inputs` with `sample_cities`. The context has been adjusted so that the state key matches the content (`city_input`) and the sub-agents extract relevant local data before formatting a tailored response.

1. Updated Agent Core (`agent.py`)

Ensure your agent definition is reading the updated variable format `{city_input}`:

Python



```

# agent.py
from google.adk.agents import LlmAgent, SequentialAgent

```

```

# Step 1: Extract landmarks and regional details based on the city string
extractor_agent = LlmAgent(
    name="CityExtractorAgent",
    model="gemini-2.5-flash",
    instruction="""
    Analyze the user's provided {city_input}.
    Identify its main geographical region, climate profile, and two famous landmarks.
    Output this information as a concise summary.
    """,
    output_key="extracted_city_data"
)

# Step 2: Formulate a travel-guide style snippet
responder_agent = LlmAgent(
    name="TravelGuideAgent",
    model="gemini-2.5-flash",
    instruction="""
    Review the structured city information inside {extracted_city_data}.
    Generate a welcoming, professional paragraph inviting tourists to visit, highlight
    """,
    output_key="final_response"
)

# Define the Pipeline
sequential_pipeline = SequentialAgent(
    name="TravelPipeline",
    sub_agents=[extractor_agent, responder_agent]
)

```

2. Updated Testing Script (test_agent.py)

This test script now iterates through `sample_cities`, dynamically isolating contexts across each loop execution.

Python



```

# test_agent.py
import asyncio
from google.adk.context import InvocationContext
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city-based pipeline test environment...\n")

    # Replaced with the target sample cities array
    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

```

```

# Loop over each city configuration
for index, city in enumerate(sample_cities, start=1):
    print(f"=====")
    print(f" RUNNING TEST CASE #{index}: {city}")
    print(f"=====")

    # Instantiate a clean context for every loop to prevent overlapping state data
    ctx = InvocationContext()
    ctx.session.state["city_input"] = city

    print("Executing Sequential Pipeline...")
    # Consume the event stream until completion
    async for event in sequential_pipeline.run_async(ctx):
        pass

    print("Pipeline Execution Completed.")

    # Retrieve context outputs
    extracted = ctx.session.state.get("extracted_city_data")
    final_output = ctx.session.state.get("final_response")

    print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
    print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")

    # Validate state mutations
    assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
    assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

    print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")

if __name__ == "__main__":
    asyncio.run(run_pipeline_test())

```

I need to provide city as a message to the sequential agent

To provide the city string as an explicit incoming user message instead of a pre-populated state variable, you need to change two things:

1. In `agent.py` : Remove the state bracket placeholder `{city_input}` from the first agent's instructions. By default, an `LlmAgent` automatically consumes the user's input message as its prompt when no explicit variables are given.

2. In `test_agent.py` : Construct a structured `Content` message object using the ADK types and pass it to the `user_content` field when initializing the `InvocationContext`.

Here is the updated configuration:

1. Updated Agent Configuration (`agent.py`)

Python



```
# agent.py
from google.adk.agents import LlmAgent, SequentialAgent

# Step 1: Receives the user message directly from the invocation context
extractor_agent = LlmAgent(
    name="CityExtractorAgent",
    model="gemini-2.5-flash",
    instruction="""
    Analyze the user's provided message containing a city.
    Identify its main geographical region, climate profile, and two famous landmarks.
    Output this information as a concise summary.
    """,
    output_key="extracted_city_data"
)

# Step 2: Formulate a travel guide snippet based on Step 1's state output
responder_agent = LlmAgent(
    name="TravelGuideAgent",
    model="gemini-2.5-flash",
    instruction="""
    Review the structured city information inside {extracted_city_data}.
    Generate a welcoming, professional paragraph inviting tourists to visit, highlight
    """,
    output_key="final_response"
)

sequential_pipeline = SequentialAgent(
    name="TravelPipeline",
    sub_agents=[extractor_agent, responder_agent]
)
```

2. Updated Testing Script (`test_agent.py`)

Here we construct `Content` and `Part` primitives from `google.adk.types` and supply them cleanly as the core user event trigger for the pipeline.

Python



```
# test_agent.py
import asyncio
from google.adk.context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        print(f"=====")
        print(f" RUNNING TEST CASE #{index}: Sending message '{city}'")
        print(f"=====")

        # 1. Package the city text into an official ADK Content message frame
        user_message = Content(
            role="user",
            parts=[Part(text=city)]
        )

        # 2. Assign the message object directly into user_content
        ctx = InvocationContext(user_content=user_message)

        print("Executing Sequential Pipeline...")
        async for event in sequential_pipeline.run_async(ctx):
            pass

        print("Pipeline Execution Completed.")

        # Retrieve context outputs from downstream steps
        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
        print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")

        assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
        assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

        print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")
```

```
if __name__ == "__main__":  
    asyncio.run(run_pipeline_test())
```

This code is written in google colab notebook. Please provide the code for all cells

Here is the complete code structured for a Google Colab notebook environment.

Since Google Colab uses a single-file runtime workspace, we will use the `%%writefile` magic command in the first cell to dynamically generate the `agent.py` file. The second cell installs the necessary dependencies, and the third cell runs your multi-city message-based pipeline test.

Cell 1: Create the Agent Module

Run this cell to write the multi-agent pipeline setup directly into the Colab workspace.

Python



```
%%writefile agent.py  
from google.adk.agents import LlmAgent, SequentialAgent  
  
# Step 1: Receives the user message directly from the invocation context  
extractor_agent = LlmAgent(  
    name="CityExtractorAgent",  
    model="gemini-2.5-flash",  
    instruction="""  
Analyze the user's provided message containing a city.  
Identify its main geographical region, climate profile, and two famous landmarks.  
Output this information as a concise summary.  
""",  
    output_key="extracted_city_data"  
)  
  
# Step 2: Formulate a travel guide snippet based on Step 1's state output  
responder_agent = LlmAgent(  
    name="TravelGuideAgent",  
    model="gemini-2.5-flash",  
    instruction="""  
Review the structured city information inside {extracted_city_data}.  
Generate a welcoming, professional paragraph inviting tourists to visit, highligh  
""",
```

```

        output_key="final_response"
    )

    sequential_pipeline = SequentialAgent(
        name="TravelPipeline",
        sub_agents=[extractor_agent, responder_agent]
    )

```

Cell 2: Install Dependencies & Setup Environment Variables

Run this cell to install the framework package. Make sure to replace

`YOUR_GEMINI_API_KEY` with your actual Google AI Studio API key so the Gemini model can authenticate.

Python



```

# Install the Google Agent Development Kit framework
!pip install google-adk

import os
# Set your Gemini API key (Required for gemini-2.5-flash)
os.environ["GEMINI_API_KEY"] = "YOUR_GEMINI_API_KEY"

```

Cell 3: Test Execution Script

Run this cell to loop through your sample cities array, feeding each city string sequentially into the agent pipeline as an official `user_content` message.

Python



```

import asyncio
from google.adk.context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        print(f"=====")
        print(f"RUNNING TEST CASE #{index}: Sending message '{city}'")

```

```

print(f"=====")

# Package the city text into an official ADK Content message frame
user_message = Content(
    role="user",
    parts=[Part(text=city)]
)

# Assign the message object directly into user_content
ctx = InvocationContext(user_content=user_message)

print("Executing Sequential Pipeline...")
async for event in sequential_pipeline.run_async(ctx):
    pass

print("Pipeline Execution Completed.")

# Retrieve context outputs from downstream steps
extracted = ctx.session.state.get("extracted_city_data")
final_output = ctx.session.state.get("final_response")

print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")

assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")

# Run the async test main loop directly in Colab
await run_pipeline_test()

```

I need to pass "What is your {city} to the agent

To dynamically format your city inputs into a question structure like "What is your review of Chicago, IL?" or simply custom-build the text string before passing it into the ADK message object, you can use a Python f-string inside the execution loop.

Here is the updated configuration for all three cells in your Google Colab notebook:

Cell 1: Create the Agent Module

(No changes needed here, but keeping it complete for consistency)

Python



```
%%writefile agent.py
from google.adk.agents import LlmAgent, SequentialAgent

# Step 1: Receives the formatted user message directly from the invocation context
extractor_agent = LlmAgent(
    name="CityExtractorAgent",
    model="gemini-2.5-flash",
    instruction="""
    Analyze the user's provided question or message containing a city.
    Identify the main geographical region, climate profile, and two famous landmarks
    Output this information as a concise summary.
    """,
    output_key="extracted_city_data"
)

# Step 2: Formulate a travel guide snippet based on Step 1's state output
responder_agent = LlmAgent(
    name="TravelGuideAgent",
    model="gemini-2.5-flash",
    instruction="""
    Review the structured city information inside {extracted_city_data}.
    Generate a welcoming, professional paragraph inviting tourists to visit, highlight
    """,
    output_key="final_response"
)

sequential_pipeline = SequentialAgent(
    name="TravelPipeline",
    sub_agents=[extractor_agent, responder_agent]
)
```

Cell 2: Install Dependencies & Setup Environment Variables

Python



```
# Install the Google Agent Development Kit framework
!pip install google-adk

import os
# Set your Gemini API key (Required for gemini-2.5-flash)
os.environ["GEMINI_API_KEY"] = "YOUR_GEMINI_API_KEY"
```

Cell 3: Test Execution Script

This cell now formats the string into your template prompt before packaging it into the `user_content` block.

Python



```
import asyncio
from google.adk.context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        # Format the city dynamically into your desired phrase structure
        formatted_message_text = f"What is your perspective on {city}?"

        print(f"=====")
        print(f" RUNNING TEST CASE #{index}")
        print(f"=====")
        print(f"Sending formatted message: '{formatted_message_text}'\n")

        # Package the dynamically constructed string into an official ADK Content fra
        user_message = Content(
            role="user",
            parts=[Part(text=formatted_message_text)]
        )

        # Assign the message object directly into user_content
        ctx = InvocationContext(user_content=user_message)

        print("Executing Sequential Pipeline...")
        async for event in sequential_pipeline.run_async(ctx):
            pass

        print("Pipeline Execution Completed.")

        # Retrieve context outputs from downstream steps
        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
        print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")
```

```
assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")

# Run the async test main loop directly in Colab
await run_pipeline_test()
```

I am getting error No module named 'google.adk.context'

That error is happening because there is no package named `google.adk.context` in the framework. In the Python version of the Google Agent Development Kit, `InvocationContext` is located inside the `google.adk.agents` module (or explicitly from `google.adk.agents.invocation_context`).

To fix the error, update your import line in **Cell 3** (`test_agent.py` script) from:

Python



```
# ✗ INCORRECT (Throws ModuleNotFoundError)
from google.adk.context import InvocationContext
```

to this:

Python



```
# CORRECT
from google.adk.agents import InvocationContext
```

Updated Full Test Script (Cell 3)

Replace the code in your test execution cell with the corrected imports below:

Python



```
import asyncio
# Corrected import paths
```

```

from google.adk.agents import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        # Format the city dynamically into your desired phrase structure
        formatted_message_text = f"What is your perspective on {city}?"

        print(f"=====")
        print(f" RUNNING TEST CASE #{index}")
        print(f"=====")
        print(f"Sending formatted message: '{formatted_message_text}'\n")

        # Package the dynamically constructed string into an official ADK Content fra
        user_message = Content(
            role="user",
            parts=[Part(text=formatted_message_text)]
        )

        # Assign the message object directly into user_content
        ctx = InvocationContext(user_content=user_message)

        print("Executing Sequential Pipeline...")
        async for event in sequential_pipeline.run_async(ctx):
            pass

        print("Pipeline Execution Completed.")

        # Retrieve context outputs from downstream steps
        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
        print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")

        assert extracted is not None, f"Pipeline failed on {city}: Extracted data is
        assert final_output is not None, f"Pipeline failed on {city}: Final response

        print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")

# Run the async test main loop directly in Colab
await run_pipeline_test()

```


I am using google-adk 1.5. I am now getting error cannot import name 'InvocationContext' from 'google.adk.agents'

My apologies for that runaround. In `google-adk 1.5`, the `InvocationContext` class resides explicitly inside its own submodule instead of being exposed directly at the top level of `google.adk.agents`.

To resolve the `ImportError`, modify the import statement in **Cell 3** to target the full submodule path:

Python



```
# CORRECT IMPORT PATH FOR ADK 1.5
from google.adk.agents.invocation_context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline
```

Complete Corrected Script (Cell 3)

Python



```
import asyncio
# Explicitly import from the invocation_context submodule
from google.adk.agents.invocation_context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        formatted_message_text = f"What is your perspective on {city}?"

        print(f"=====")
        print(f" RUNNING TEST CASE #{index}")
        print(f"=====")
        print(f"Sending formatted message: '{formatted_message_text}'\n")
```

```

user_message = Content(
    role="user",
    parts=[Part(text=formatted_message_text)]
)

# Supply the content directly into the context constructor
ctx = InvocationContext(user_content=user_message)

print("Executing Sequential Pipeline...")
async for event in sequential_pipeline.run_async(ctx):
    pass

print("Pipeline Execution Completed.")

extracted = ctx.session.state.get("extracted_city_data")
final_output = ctx.session.state.get("final_response")

print(f"\n[Step 1 Output] Extracted Region & Landmark Profile:\n{extracted}")
print(f"\n[Step 2 Output] Formatted Travel Suggestion:\n{final_output}")

assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

print(f"\nTest Case #{index} ({city}) Status: SUCCESS\n")

# Await execution in the Google Colab environment
await run_pipeline_test()

```

How do I print text and non-text response from the agents in `run_pipeline_test` using `display` and `Markdown` objects

To render the model's responses in Google Colab with rich visual formatting (like bold headings, bullet points, and syntax highlighting) instead of raw text strings, you can use `IPython.display` objects.

When a model replies using the Google ADK, it frequently returns text responses but can also return structured data, JSON blobs, or complex content. Converting these outputs to markdown makes them significantly easier to read in notebook cells.

Here is the updated **Cell 3** code. It imports `display` and `Markdown` and alters the print sections to use clean notebook layouts.

Updated Testing Script (Cell 3)

Python



```
import asyncio
from google.adk.agents.invocation_context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline

# Import Colab's native rich display utilities
from IPython.display import display, Markdown

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        formatted_message_text = f"What is your perspective on {city}?"

        # Display a clean structural markdown header for each test case
        display(Markdown(f"---"))
        display(Markdown(f"## 🚧 RUNNING TEST CASE #{index}"))
        display(Markdown(f"**Sent Message:** `{{formatted_message_text}}`"))

        user_message = Content(
            role="user",
            parts=[Part(text=formatted_message_text)]
        )

        ctx = InvocationContext(user_content=user_message)

        print("Executing Sequential Pipeline...")
        async for event in sequential_pipeline.run_async(ctx):
            pass

        print("Pipeline Execution Completed.")

        # Extract response state
        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        # Render the responses cleanly using Markdown blocks
        display(Markdown(f"### 🔍 Step 1 Output: Extracted Region & Landmark Profile"))
        # If the output might look like a dictionary or JSON block, wrapping it in ``
        display(Markdown(f"```json\n{{extracted}}\n```" if isinstance(extracted, (dict,

        display(Markdown(f"### 📝 Step 2 Output: Formatted Travel Suggestion"))
```

```
display(Markdown(f"{final_output}"))

# Assertions
assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

display(Markdown(f"**Status:** ✅ Test Case #{index} ({city}) Success\n"))

# Run the async execution loop directly in Colab
await run_pipeline_test()
```

What This Does:

1. `display(Markdown("..."))` : Instructs Colab to evaluate the string argument as genuine Markdown. Headers (`##` , `###`), bolding (`bold`), and inline backticks are parsed and rendered visually.
2. **Dynamic Code Fencing:** For **Step 1**, if the agent outputs raw text string data, it renders normally. If it outputs structured JSON text data, wrapping it inside a `````

Change the above code to extract every event

To capture and process every event as the pipeline streams its execution, you can inspect the `event` variable inside the `async for` loop. In `google-adk`, these events are structural objects that contain tracking information (like step status updates, tool calls, or content chunks).

By rendering these events using Colab's `display` and `Markdown` objects, you can track the lifecycle of each sub-agent in real time.

Here is the updated **Cell 3** code containing the event tracking loop:

Updated Testing Script (Cell 3)

Python



```
import asyncio
from google.adk.agents.invocation_context import InvocationContext
from google.adk.types import Content, Part
from agent import sequential_pipeline
```

```

# Import Colab's native rich display utilities
from IPython.display import display, Markdown

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        formatted_message_text = f"What is your perspective on {city}?"

        display(Markdown(f"---"))
        display(Markdown(f"### 🚧 RUNNING TEST CASE #{index}"))
        display(Markdown(f"**Sent Message:** `{formatted_message_text}`"))

        user_message = Content(
            role="user",
            parts=[Part(text=formatted_message_text)]
        )

        ctx = InvocationContext(user_content=user_message)

        display(Markdown("### 📡 Real-Time Pipeline Events"))

        # Stream the sequential agent execution and extract every event
        async for event in sequential_pipeline.run_async(ctx):
            # Check the structural type or representation of the event object
            event_type = type(event).__name__

            # Format and display the live event
            display(Markdown(f"#### ⬢ Event Triggered: `{event_type}`"))

            # If the event has readable text content or step metadata, extract it safely
            if hasattr(event, 'content') and event.content:
                display(Markdown(f"> **Content Delta:** {event.content}"))
            elif hasattr(event, 'step') and event.step:
                display(Markdown(f"> **Pipeline Step:** Running agent `{event.step.agent}`"))
            else:
                # Fallback to structural string representation for internal state details
                display(Markdown(f"`python\n{repr(event)}\n`"))

        display(Markdown("### 🏁 Pipeline Execution Completed."))

        # Extract finalized response states
        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        # Render the final summaries using Markdown blocks
        display(Markdown("### 🔍 Final Extracted Region & Landmark Profile"))

```

```

display(Markdown(f"```\njson\n{extracted}\n```" if isinstance(extracted, (dict,

display(Markdown("### 📝 Final Formatted Travel Suggestion"))
display(Markdown(f"{final_output}"))

# Assertions
assert extracted is not None, f"Pipeline failed on {city}: Extracted data is
assert final_output is not None, f"Pipeline failed on {city}: Final response

display(Markdown(f"**Status:** ✅ Test Case #{index} ({city}) Success\n"))

# Run the async execution loop directly in Colab
await run_pipeline_test()

```

What Changed?

- **Live Event Inspector:** The `async` for event in `sequential_pipeline.run_async(ctx)` loop now dynamically intercepts every chunk and milestone emitted by the ADK framework.
- **hasattr Checking:** Because events vary structurally based on whether they represent an execution state transition or text streaming delta chunks, the code safely uses `hasattr` to discover and parse fields like `.content` or `.step` without throwing attribute errors.

It is giving error No module named 'google.adk.types

The error `ModuleNotFoundError: No module named 'google.adk.types'` occurs because `google-adk` does not have an internal module named `.types`.

Under the hood, `google-adk` relies entirely on the modern `google-genai` SDK for its underlying content, message formats, and data models. Therefore, structural components like `Content` and `Part` are actually imported directly from `google.genai.types`.

To fix this, update your imports in **Cell 3** as follows.

Updated Testing Script (Cell 3)

Python



```

import asyncio
from google.adk.agents.invocation_context import InvocationContext

# ✅ FIXED: Import Content and Part from the underlying google.genai.types module
from google.genai.types import Content, Part

from agent import sequential_pipeline
from IPython.display import display, Markdown

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

    for index, city in enumerate(sample_cities, start=1):
        formatted_message_text = f"What is your perspective on {city}?"

        display(Markdown(f"---"))
        display(Markdown(f"### 🚧 RUNNING TEST CASE #{index}"))
        display(Markdown(f"**Sent Message:** `_{formatted_message_text}_`"))

        # Package using the correct GenAI content models
        user_message = Content(
            role="user",
            parts=[Part(text=formatted_message_text)]
        )

        ctx = InvocationContext(user_content=user_message)

        display(Markdown("### 🚦 Real-Time Pipeline Events"))

        async for event in sequential_pipeline.run_async(ctx):
            event_type = type(event).__name__
            display(Markdown(f"#### ⬢ Event Triggered: `_{event_type}_`"))

            if hasattr(event, 'content') and event.content:
                display(Markdown(f"> **Content Delta:** {event.content}"))
            elif hasattr(event, 'step') and event.step:
                display(Markdown(f"> **Pipeline Step:** Running agent `_{event.step.ag`"))
            else:
                display(Markdown(f"`python\n{repr(event)}\n`"))

        display(Markdown("### 🏁 Pipeline Execution Completed."))

        extracted = ctx.session.state.get("extracted_city_data")
        final_output = ctx.session.state.get("final_response")

        display(Markdown("### 🔍 Final Extracted Region & Landmark Profile"))
        display(Markdown(f"`json\n{extracted}\n`" if isinstance(extracted, (dict,

```



```
display(Markdown("### 📄 Final Formatted Travel Suggestion"))
display(Markdown(f"{final_output}"))

assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

display(Markdown(f"**Status:** ✅ Test Case #{index} ({city}) Success\n"))

# Run execution loop inside Colab
await run_pipeline_test()
```

What Changed?

- `from google.genai.types import Content, Part`: Because `google-adk` passes message histories straight into the underlying Gemini models, it skips re-inventing text objects and uses the official `google-genai` types instead.

To see a complete visual walk-through of the directory setups, configuration variables, and multi-agent systems, check out this [Google ADK Multi-Agent Tutorial](#). This video breaks down how to orchestrate specialized agents alongside memory and external tool dependencies.

📺 Google ADK Tutorial: Build Multi-Agent AI Systems from Scratch - YouTube
KodeKloud · 41K views



I am not getting any output from the print statements. Does InvocationContext API work? Should I use LLMRequest instead?

Yes, the `InvocationContext` API absolutely works—it is the core runtime brain of the entire Google ADK framework. You should **not** switch to `LLMRequest`, because `LLMRequest` is an internal model low-level data frame used inside agent lifecycles/callbacks, not something meant for triggering an agent orchestration loop.

The reason you aren't seeing any printed logs inside your event loop is a common nuance with **how async event generators behave when an error or authentication failure happens silently** during the initialization phase of `run_async()`.

If your Gemini API key is missing or invalid, or if the model name is mismatched, the underlying async generator doesn't emit data chunks—it breaks before entering the `async for` block, causing it to skip your print statements entirely.

Here is exactly how to fix it and force Colab to show you what is going on:

1. Add Error Handling to your Test Loop

Wrap your `async for` block in a `try/except` block. This ensures that if the Gemini model fails to initialize or authenticate, the error is caught and printed out rather than failing silently.

Python



```
# ... [Your previous context setups] ...
ctx = InvocationContext(user_content=user_message)

display(Markdown("### 🚀 Real-Time Pipeline Events"))

try:
    # Stream the sequential agent execution and extract every event
    async for event in sequential_pipeline.run_async(ctx):
        event_type = type(event).__name__
        display(Markdown(f"#### ♦ Event Triggered: `{event_type}`"))

        if hasattr(event, 'content') and event.content:
            display(Markdown(f"> **Content Delta:** {event.content}"))
        elif hasattr(event, 'step') and event.step:
            display(Markdown(f"> **Pipeline Step:** Running agent `{event.step}`"))
        else:
            pass
```

```
display(Markdown(f"```python\n{repr(event)}\n```"))
except Exception as e:
    # 💡 THIS WILL CAPTURE WHY YOU ARE GETTING NO OUTPUT
    display(Markdown(f"### ❌ CRITICAL RUNTIME ERROR:\n`{str(e)}`"))
```

2. Verify Your API Key Registration

In Google Colab, executing `os.environ["GEMINI_API_KEY"] = "YOUR_KEY"` works, but it can be brittle if the cell wasn't run or got cleared.

The most secure way to handle this in Colab is using the **Colab Secrets (Key icon)** in the left sidebar:

1. Click the  icon on the left panel.
2. Add a new secret named `GEMINI_API_KEY`.
3. Paste your actual Gemini API key from Google AI Studio.
4. Toggle "Notebook access" to on.

Then, update **Cell 2** to pull it securely:

Python



```
import os
from google.colab import userdata

# Safely extract the secret directly from the Colab environment
os.environ["GEMINI_API_KEY"] = userdata.get('GEMINI_API_KEY')
```

3. Double-Check your Model Name

Ensure that you are using a supported model. In `agent.py`, we used `gemini-2.5-flash`. If your project environment or underlying package configuration relies on a different workspace generation, you can fallback to the standard production moniker:

Python



```
model="gemini-1.5-flash" # or gemini-2.5-flash depending on your tier
```

I am still not seeing any output

I see exactly what is causing the silence now. In the Google Agent Development Kit framework, **you cannot call `run_async()` directly on an Agent object**. Instead, agents must be passed into an ADK `Runner`, which is responsible for initializing the framework environment, binding your agent orchestrations, managing session memory, and executing the async event generator pipeline.

Because we bypassed the `Runner`, the generator returned empty and your code skipped over the loop without displaying any logs or hitting the `try/except` block.

Here is the fully revised, production-ready **Cell 3** that fixes this by using the `Runner` and an `InMemorySessionService`.

Updated Testing Script (Cell 3)

Python



```
import asyncio
from google.adk.agents.invocation_context import InvocationContext
from google.genai.types import Content, Part
from agent import sequential_pipeline

# Import the necessary Runner and Session primitives from ADK
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Import Colab's native rich display utilities
from IPython.display import display, Markdown

async def run_pipeline_test():
    print("Initializing city pipeline message-based test environment...\n")

    # 1. Initialize the required underlying ADK session services
    session_service = InMemorySessionService()

    # 2. Attach your sequential pipeline to the official ADK Execution Runner
    runner = Runner(
        agent=sequential_pipeline,
        app_name="travel_pipeline_app",
        session_service=session_service
    )
```

```

sample_cities = ["Chicago, IL", "Miami, FL", "Seattle, WA"]

for index, city in enumerate(sample_cities, start=1):
    formatted_message_text = f"What is your perspective on {city}?"

    display(Markdown(f"---"))
    display(Markdown(f"## 🏃 RUNNING TEST CASE #{index}"))
    display(Markdown(f"**Sent Message:** `formatted_message_text`"))

    # Package using the correct GenAI content models
    user_message = Content(
        role="user",
        parts=[Part(text=formatted_message_text)]
    )

    # Define static identifiers for tracking session instances across execution
    user_id = f"test_user_{index}"
    session_id = f"session_{index}"

    display(Markdown("### 🚦 Real-Time Pipeline Events"))

    try:
        # ✅ CORRECT WAY: Invoke run_async directly through the Runner
        async for event in runner.run_async(
            user_id=user_id,
            session_id=session_id,
            new_message=user_message
        ) or []:
            event_type = type(event).__name__
            display(Markdown(f"#### ⬢ Event Triggered: `{event_type}`"))

            # Check for streaming text content or internal stage changes
            if hasattr(event, 'content') and event.content:
                display(Markdown(f"> **Content Delta:** {event.content}"))
            elif hasattr(event, 'step') and event.step:
                display(Markdown(f"> **Pipeline Step:** Running agent `{event.step}`"))
            else:
                display(Markdown(f"`python\n{repr(event)}\n`"))

    except Exception as e:
        display(Markdown(f"### ❌ RUNTIME ERROR:\n`str(e)`"))
        continue

    display(Markdown("### 🏁 Pipeline Execution Completed."))

    # 3. Retrieve the updated session snapshot to review state variables
    active_session = await session_service.get_session(
        app_name="travel_pipeline_app",
        user_id=user_id,

```

```

        session_id=session_id
    )

    extracted = active_session.state.get("extracted_city_data")
    final_output = active_session.state.get("final_response")

    display(Markdown("### 🔍 Final Extracted Region & Landmark Profile"))
    display(Markdown(f"```json\n{extracted}\n```" if isinstance(extracted, (dict,
                                                                    list)) else ""))

    display(Markdown("### 📝 Final Formatted Travel Suggestion"))
    display(Markdown(f"{final_output}"))

    assert extracted is not None, f"Pipeline failed on {city}: Extracted data is None"
    assert final_output is not None, f"Pipeline failed on {city}: Final response is None"

    display(Markdown(f"**Status:** ✅ Test Case #{index} ({city}) Success\n"))

# Run execution loop inside Colab
await run_pipeline_test()

```